```python
import numpy as np

student_scores = np.array([
    [80, 75, 85, 70],
    [90, 85, 80, 75],
    [70, 80, 90, 85],
    [85, 90, 95, 80]
])

subjects = ["Math", "Science", "English", "History"]

average = np.mean(student_scores, axis=0)

for i in range(4):
    print(subjects[i], "Average =", average[i])

highest = np.argmax(average)
print("Highest average subject:", subjects[highest])
```

```
Math Average = 81.25
Science Average = 82.5
English Average = 87.5
History Average = 77.5
Highest average subject: English
```

```python
import numpy as np

prices = np.array([
    [100, 120, 150],
    [80, 90, 110],
    [200, 180, 160]
])

average_price = np.mean(prices)

print("Average price =", average_price)
```

```
Average price = 132.2222222222223
```

```python
import numpy as np

# Columns: Bedrooms, Area, Sale Price
house_data = np.array([
    [3, 1200, 250000],
    [5, 1800, 400000],
    [6, 2200, 500000],
    [4, 1500, 320000],
    [5, 2000, 450000]
])

selected = house_data[house_data[:, 0] > 4]
average_price = np.mean(selected[:, 2])

print("Average sale price =", average_price)
```

```
Average sale price = 450000.0
```

```python
import numpy as np

sales_data = np.array([10000, 12000, 15000, 18000])

total_sales = np.sum(sales_data)
percentage_increase = ((sales_data[3] - sales_data[0])
                       / sales_data[0]) * 100

print("Total sales =", total_sales)
print("Percentage increase =", percentage_increase, "%")
```

```
Total sales = 55000
Percentage increase = 80.0 %
```

```python
import numpy as np

fuel_efficiency = np.array([25, 30, 28, 35, 32])

average = np.mean(fuel_efficiency)

old_model = fuel_efficiency[0]
new_model = fuel_efficiency[1]

improvement = ((new_model - old_model) / old_model) * 100

print("Average fuel efficiency =", average)
print("Percentage improvement =", improvement, "%")
```

```
Average fuel efficiency = 30.0
Percentage improvement = 20.0 %
```

```python
prices = [100, 50, 200]
quantities = [2, 3, 1]

discount_rate = 10
tax_rate = 5

subtotal = 0

for price, quantity in zip(prices, quantities):
    subtotal += price * quantity

discount = subtotal * discount_rate / 100
after_discount = subtotal - discount

tax = after_discount * tax_rate / 100
total = after_discount + tax

print("Subtotal =", subtotal)
print("Discount =", discount)
print("Tax =", tax)
print("Total cost =", total)
```

```
Subtotal = 550
Discount = 55.0
Tax = 24.75
Total cost = 519.75
```

```python
import pandas as pd

order_data = pd.DataFrame({
    "Customer_ID": [1, 1, 2, 2, 3],
    "Order_Date": ["2026-01-02", "2026-01-05", "2026-01-03",
                   "2026-01-10", "2026-01-08"],
    "Product": ["Pen", "Book", "Pen", "Bag", "Book"],
    "Quantity": [2, 3, 5, 1, 4]
})

print("Orders by customer:")
print(order_data.groupby("Customer_ID").size())

print("\nAverage quantity by product:")
print(order_data.groupby("Product")["Quantity"].mean())

order_data["Order_Date"] = pd.to_datetime(order_data["Order_Date"])
print("\nEarliest date:", order_data["Order_Date"].min())
print("Latest date:", order_data["Order_Date"].max())
```

```
Orders by customer:
Customer_ID
1    2
2    2
3    1
dtype: int64

Average quantity by product:
Product
Bag     1.0
Book    3.5
Pen     3.5
Name: Quantity, dtype: float64

Earliest date: 2026-01-02 00:00:00
Latest date: 2026-01-10 00:00:00
```

```python
import pandas as pd

sales = pd.DataFrame({
    "Product": ["Pen", "Book", "Pen", "Bag", "Book", "Pen", "Bag"],
    "Quantity": [10, 8, 15, 5, 12, 20, 9]
})

top5 = sales.groupby("Product")["Quantity"].sum()
top5 = top5.sort_values(ascending=False).head(5)

print(top5)
```

```
Product
Pen     45
Book    20
Bag     14
Name: Quantity, dtype: int64
```

```python
import pandas as pd

property_data = pd.DataFrame({
    "Location": ["Chennai", "Chennai", "Madurai", "Madurai", "Chennai"],
    "Bedrooms": [3, 5, 4, 6, 5],
    "Area": [1200, 1800, 1500, 2200, 2000],
    "Price": [300000, 450000, 350000, 500000, 480000]
})

print("Average price by location:")
print(property_data.groupby("Location")["Price"].mean())

print("Properties with more than 4 bedrooms:")
print((property_data["Bedrooms"] > 4).sum())

largest = property_data.loc[property_data["Area"].idxmax()]
print("Property with largest area:")
print(largest)
```

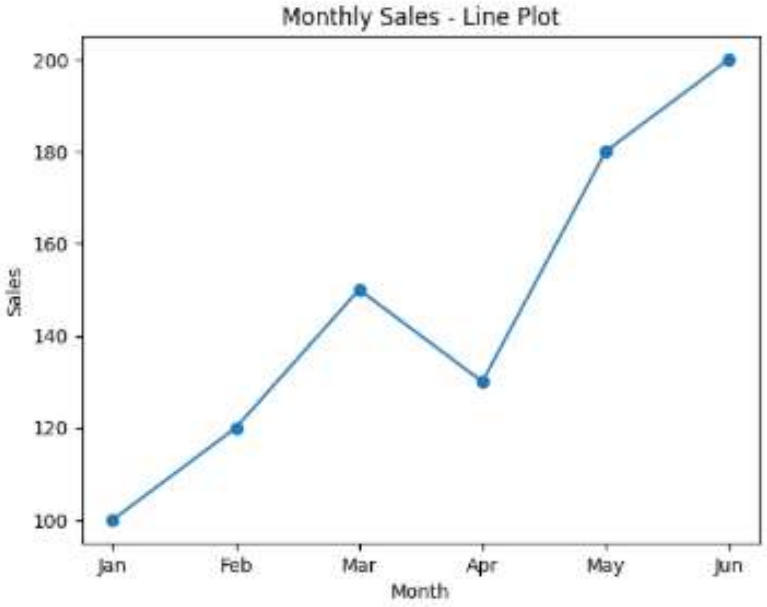
```
Average price by location:
Location
Chennai    410000.0
Madurai    425000.0
Name: Price, dtype: float64
Properties with more than 4 bedrooms:
3
Property with largest area:
Location    Madurai
Bedrooms          6
Area           2200
Price        500000
Name: 3, dtype: object
```

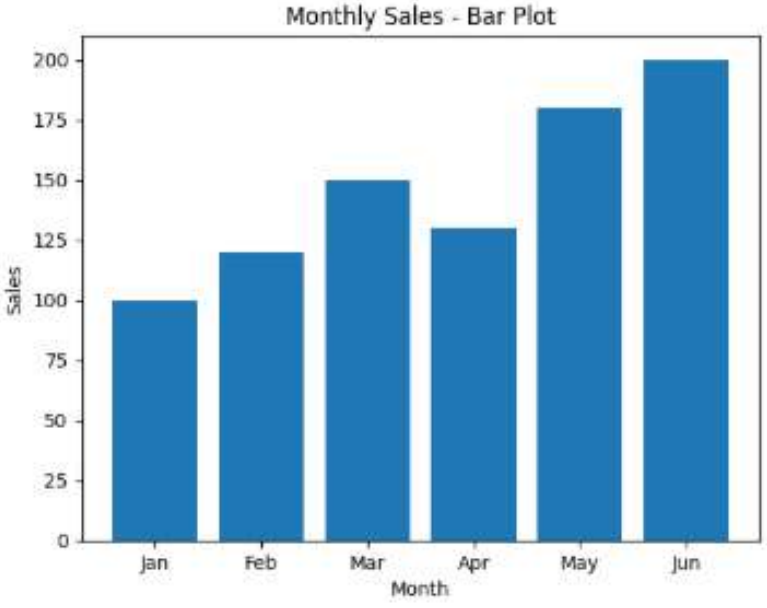
```python
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
sales = [100, 120, 150, 130, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Monthly Sales - Line Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```





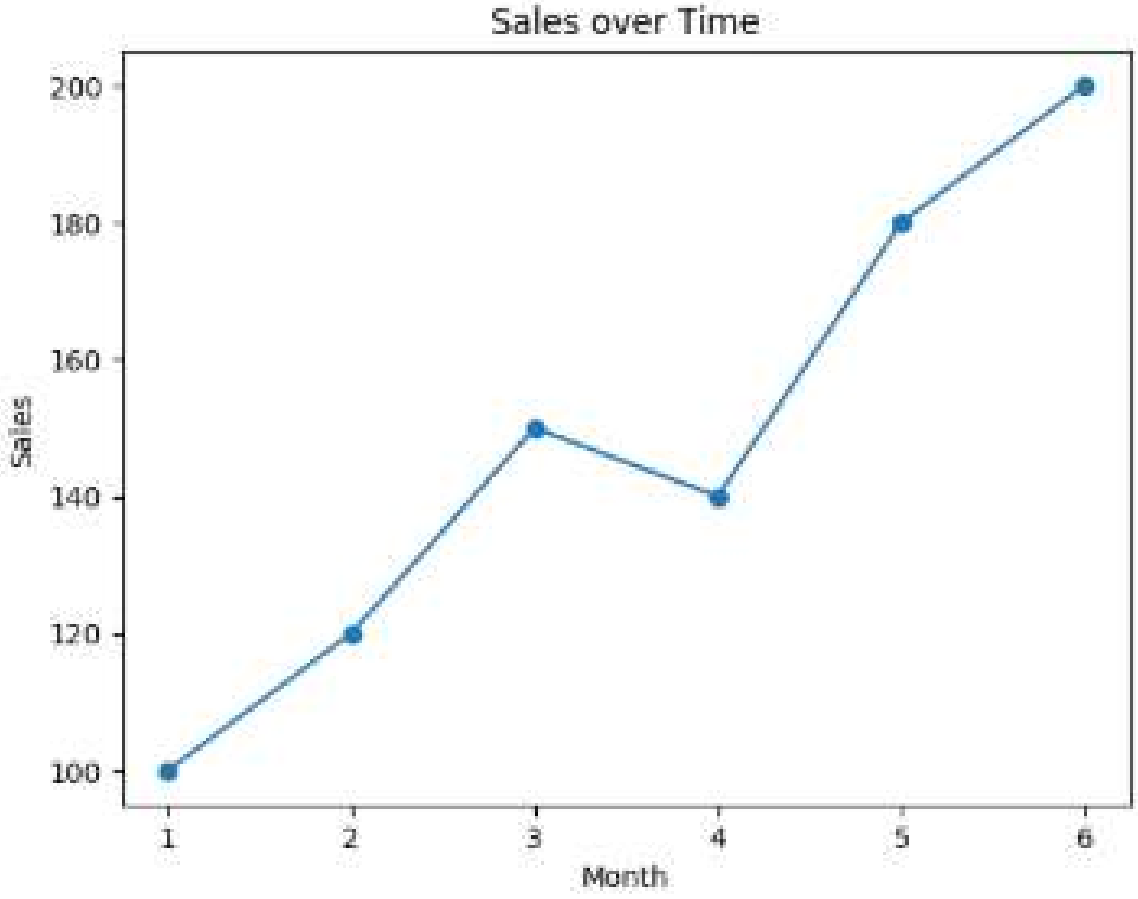
```python
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5, 6]
sales = [100, 120, 150, 140, 180, 200]

plt.plot(months, sales, marker="o")
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```





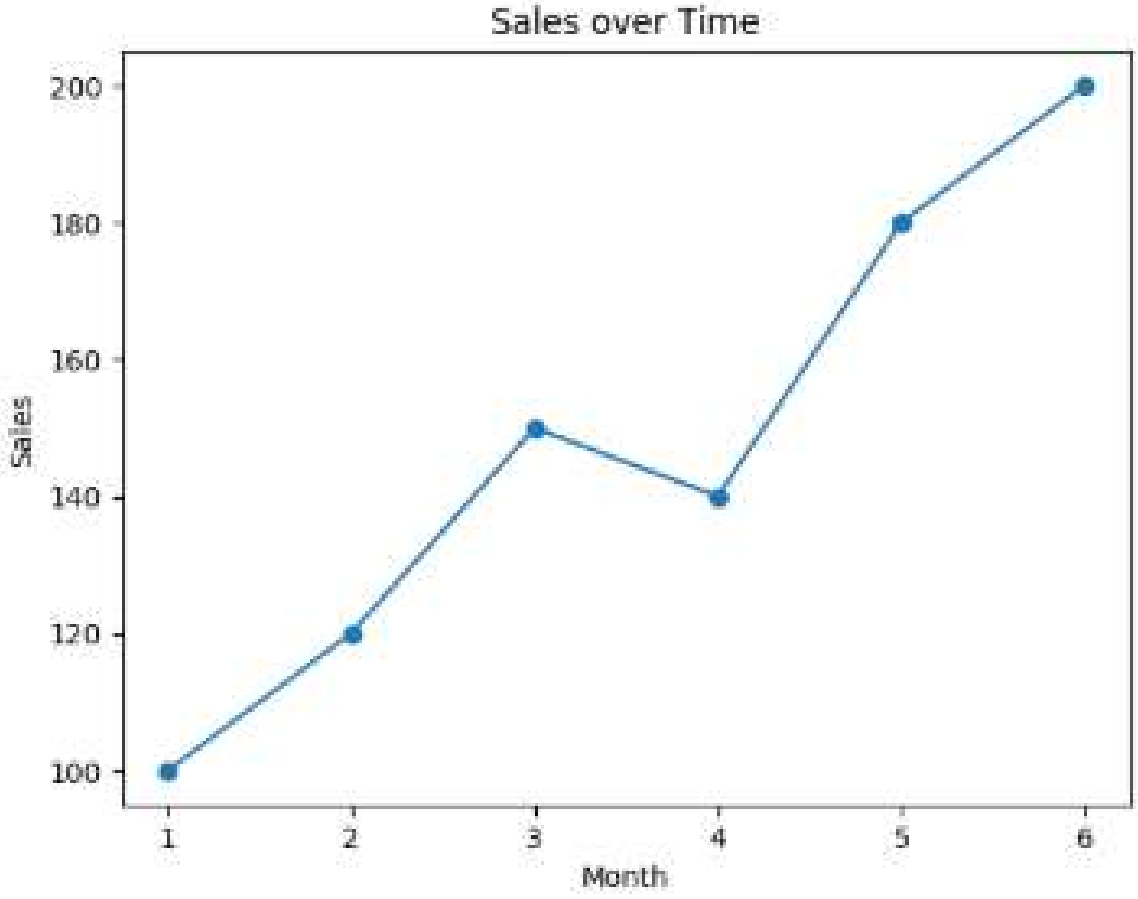
```python
import matplotlib.pyplot as plt

months = [1, 2, 3, 4, 5, 6]
sales = [100, 120, 150, 140, 180, 200]

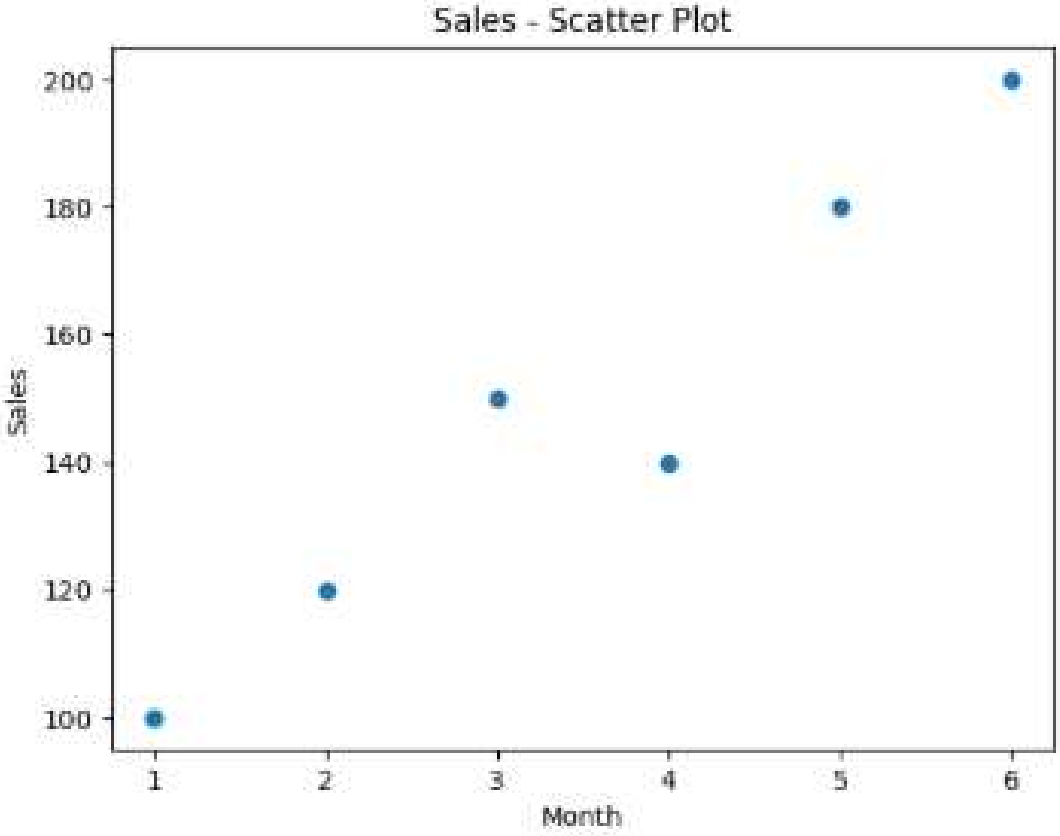
plt.plot(months, sales, marker="o")
plt.title("Sales over Time")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

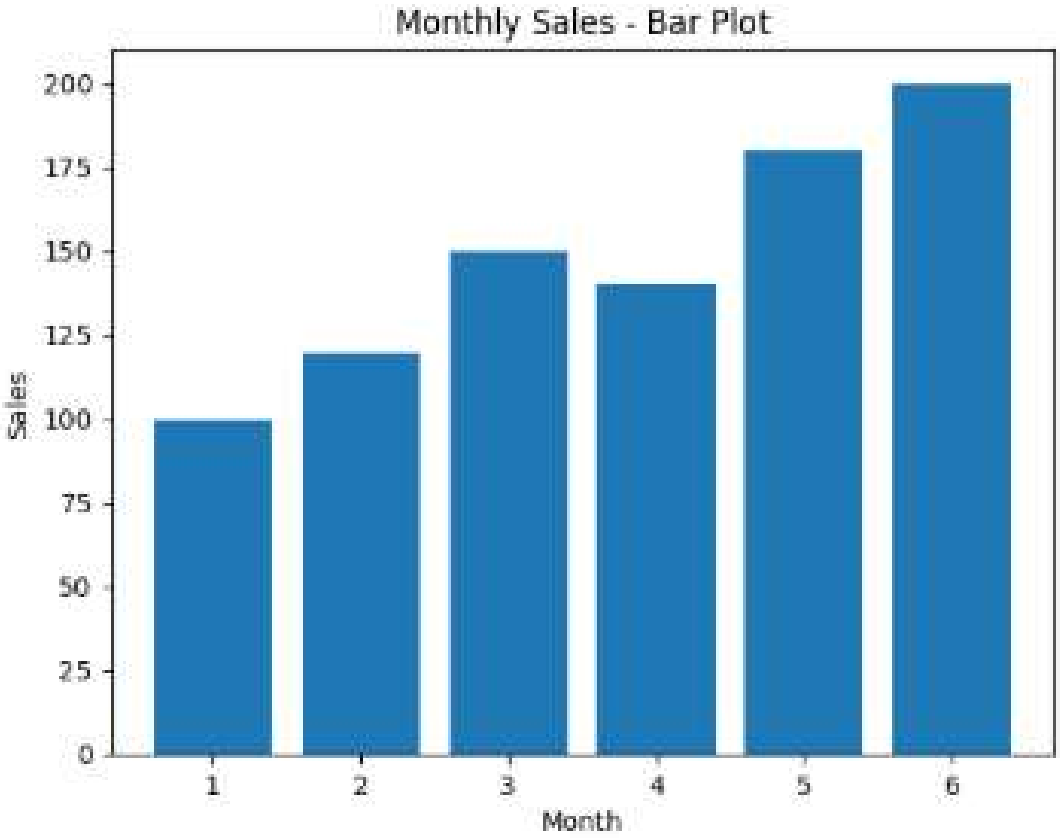
plt.scatter(months, sales)
plt.title("Sales - Scatter Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()

plt.bar(months, sales)
plt.title("Monthly Sales - Bar Plot")
plt.xlabel("Month")
plt.ylabel("Sales")
plt.show()
```






Sales - Scatter Plot
200
180
160
140
120
100
Sales
1
2
3
4
5
6
Month

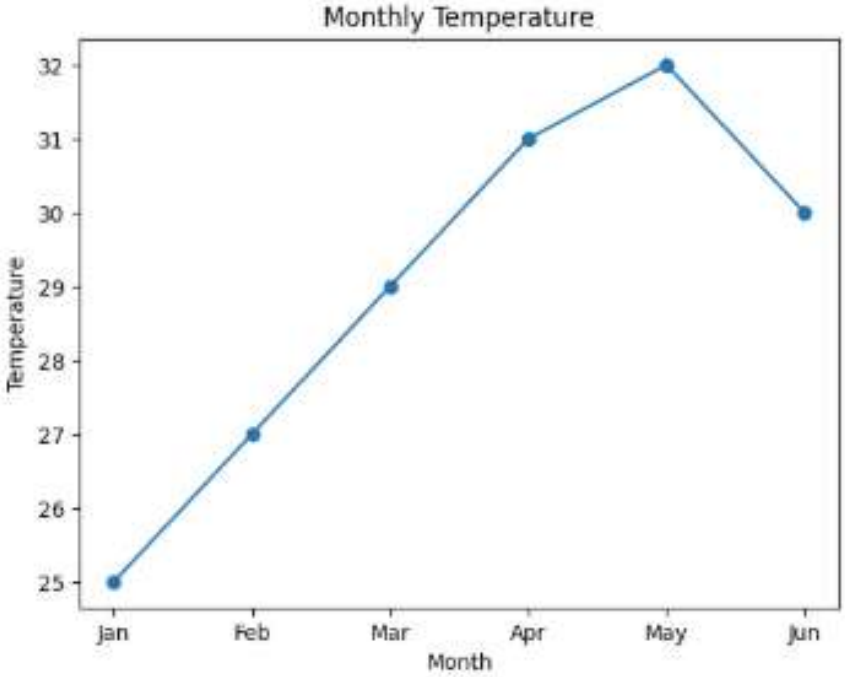

Monthly Sales - Bar Plot
200
175
150
125
100
75
50
25
0
Sales
1
2
3
4
5
6
Month

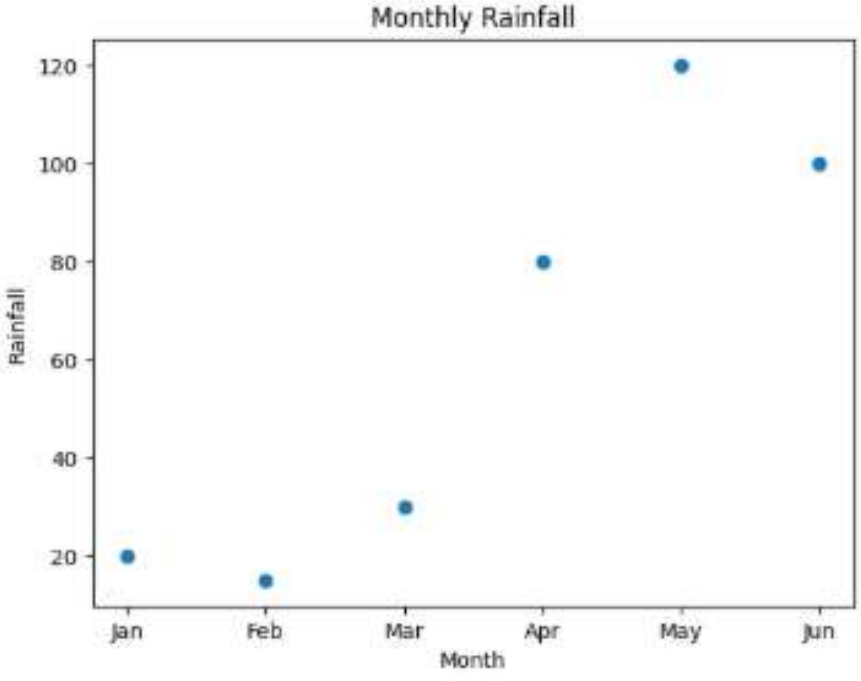
```python
import matplotlib.pyplot as plt

months = ["Jan", "Feb", "Mar", "Apr", "May", "Jun"]
temperature = [25, 27, 29, 31, 32, 30]
rainfall = [20, 15, 30, 80, 120, 100]

plt.plot(months, temperature, marker="o")
plt.title("Monthly Temperature")
plt.xlabel("Month")
plt.ylabel("Temperature")
plt.show()

plt.scatter(months, rainfall)
plt.title("Monthly Rainfall")
plt.xlabel("Month")
plt.ylabel("Rainfall")
plt.show()
```





```python
from collections import Counter
import re

# Programmatically create a sample text file to prevent FileNotFoundError
sample_content = """This is a sample text file.
This file contains sample text to test the word counter code.
Sample text files are useful for demonstrations."""

with open("sample_text.txt", "w") as file:
    file.write(sample_content)

# Read the newly created file and count word frequencies
with open("sample_text.txt", "r") as file:
    text = file.read().lower()

words = re.findall(r"\b\w+\b", text)

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

```
sample : 3
text : 3
this : 2
file : 2
is : 1
a : 1
contains : 1
to : 1
test : 1
the : 1
word : 1
counter : 1
code : 1
files : 1
are : 1
useful : 1
for : 1
demonstrations : 1
```

```python
import pandas as pd
import matplotlib.pyplot as plt

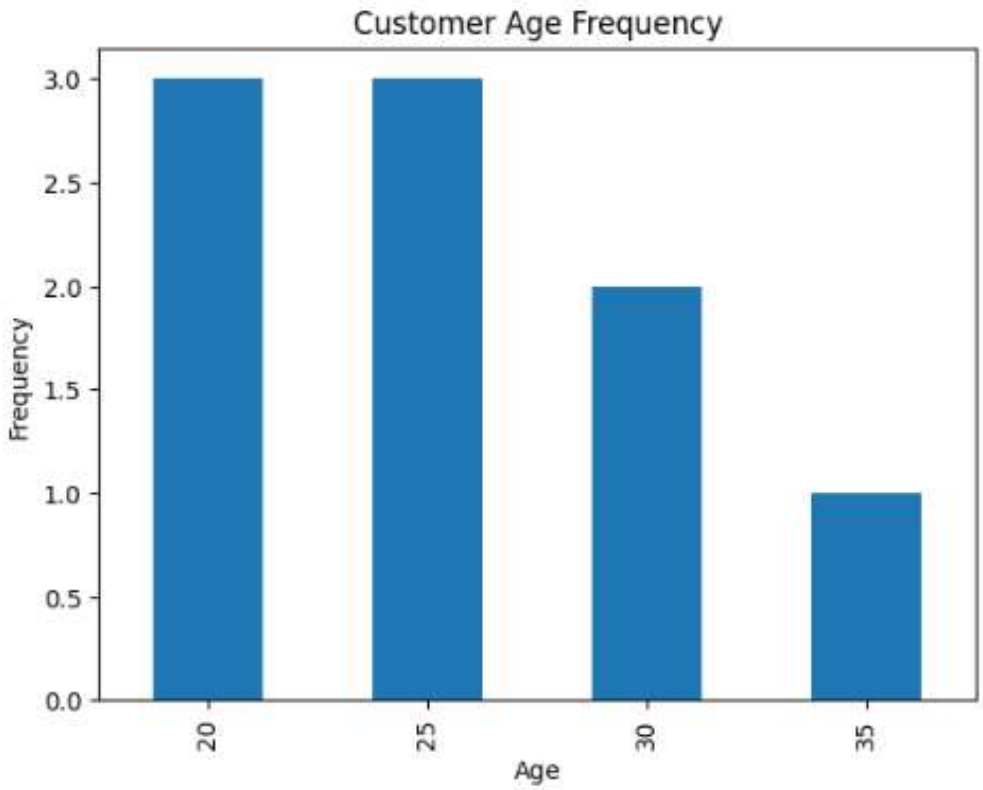
data = pd.DataFrame({
    "Age": [20, 25, 20, 30, 25, 20, 35, 30, 25]
})

frequency = data["Age"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Customer Age Frequency")
plt.xlabel("Age")
plt.ylabel("Frequency")
plt.show()
```

```
Age
20    3
25    3
30    2
35    1
Name: count, dtype: int64
```



```python
import pandas as pd
import matplotlib.pyplot as plt

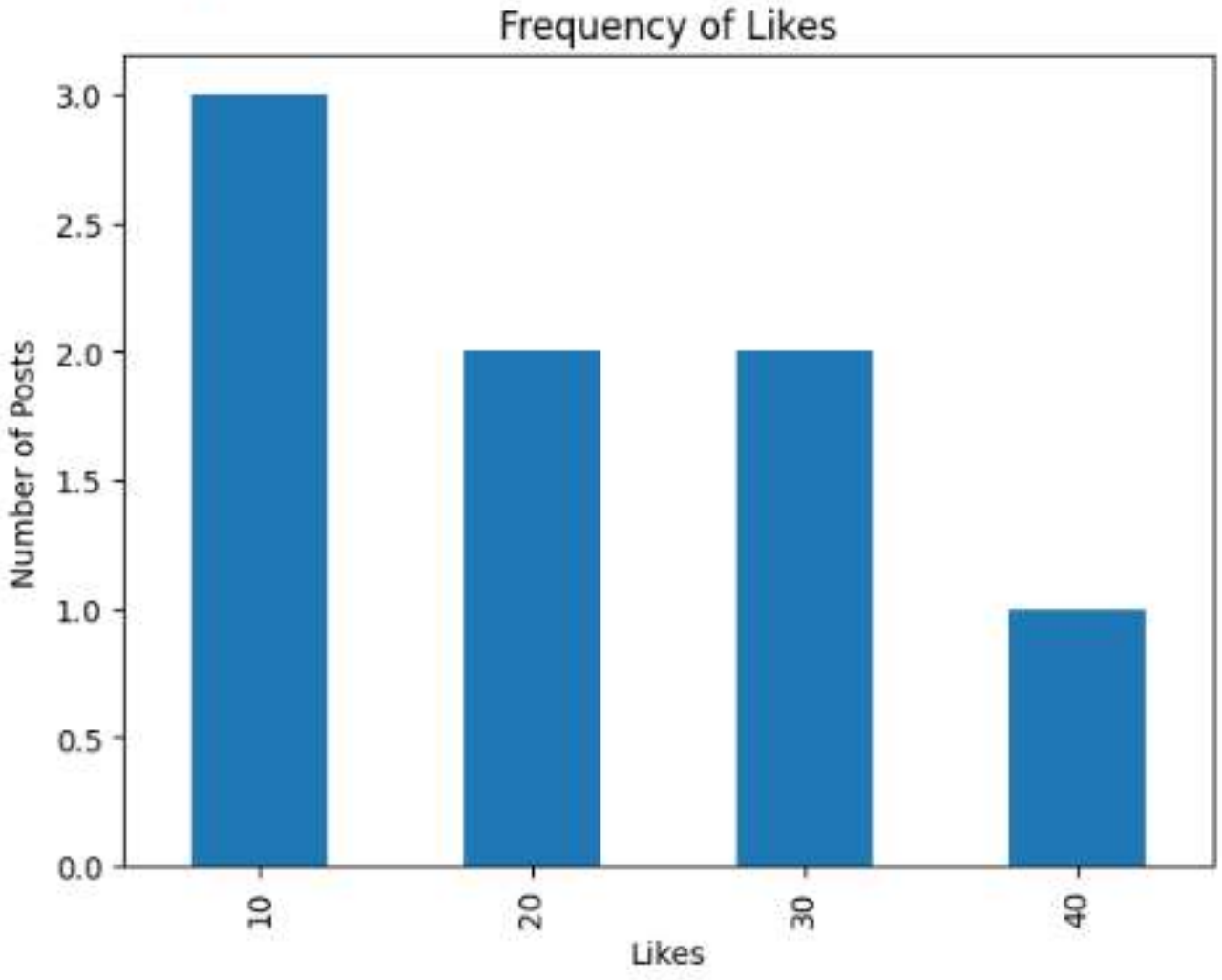
data = pd.DataFrame({
    "Likes": [10, 20, 10, 30, 20, 10, 40, 30]
})

frequency = data["Likes"].value_counts().sort_index()

print(frequency)

frequency.plot(kind="bar")
plt.title("Frequency of Likes")
plt.xlabel("Likes")
plt.ylabel("Number of Posts")
plt.show()
```

```
Likes
10    3
20    2
30    2
40    1
Name: count, dtype: int64
```



```python
import pandas as pd
import re
from collections import Counter

reviews = pd.DataFrame({
    "Review": [
        "good product and good service",
        "excellent product",
        "good quality product"
    ]
})

words = []

for review in reviews["Review"]:
    words.extend(re.findall(r"\b\w+\b", review.lower()))

frequency = Counter(words)

for word, count in frequency.most_common():
    print(word, ":", count)
```

```
good : 3
product : 3
and : 1
service : 1
excellent : 1
quality : 1
```

```python
import pandas as pd
import re
from collections import Counter
import matplotlib.pyplot as plt

# Programmatically create a mock data.csv file to resolve the FileNotFoundError
mock_data = pd.DataFrame({
    "feedback": [
        "The product is excellent and the service is great.",
        "Good quality, but shipping was a bit slow.",
        "Excellent customer service and very good quality product!",
        "Not happy with the product. The quality is poor.",
        "Amazing service! Highly recommended and very helpful support."
    ]
})
mock_data.to_csv("data.csv", index=False)

# Read the newly created dataset
data = pd.read_csv("data.csv")

stop_words = {"the", "and", "is", "a", "an", "of", "to", "in"}

words = []

for text in data["feedback"].dropna():
    text = text.lower()
    words.extend(re.findall(r"\b[a-z]+\b", text))

words = [word for word in words if word not in stop_words]

frequency = Counter(words)

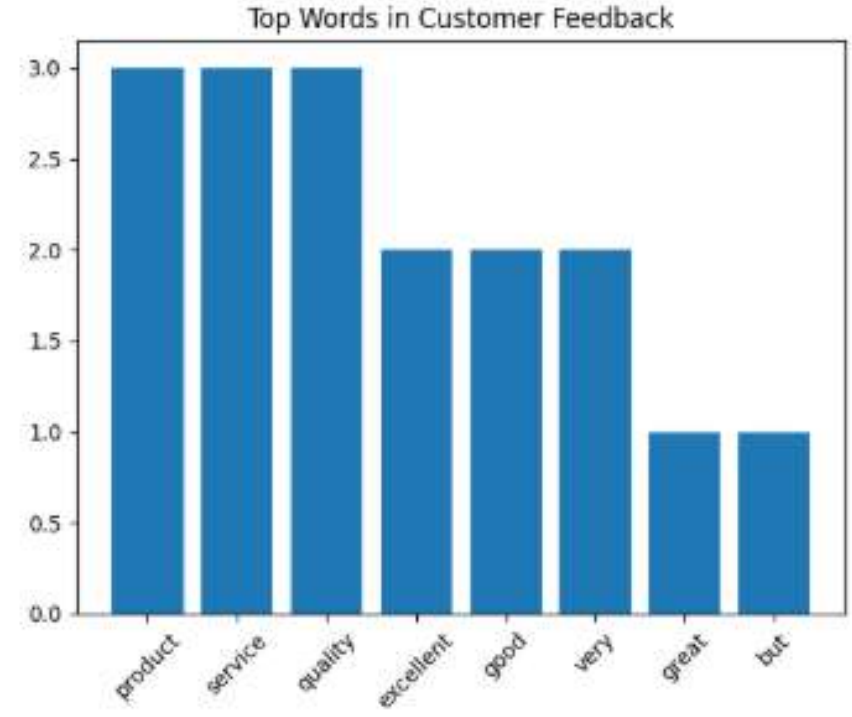
n = int(input("Enter N: "))
top_words = frequency.most_common(n)

print(top_words)

names = [x[0] for x in top_words]
counts = [x[1] for x in top_words]

plt.bar(names, counts)
plt.xticks(rotation=45)
plt.title("Top Words in Customer Feedback")
plt.show()
```

```
Enter N: 8
[('product', 3), ('service', 3), ('quality', 3), ('excellent', 2), ('good', 2), ('very', 2), ('great', 1), ('but', 1)]
```



```python
import pandas as pd
import matplotlib.pyplot as plt
import scipy.stats as stats

data = pd.DataFrame({
    "Age": [20, 22, 25, 28, 30, 32, 35, 40],
    "Fat": [15, 18, 20, 22, 24, 27, 30, 32]
})

print("Mean:")
print(data.mean())

print("Median:")
print(data.median())

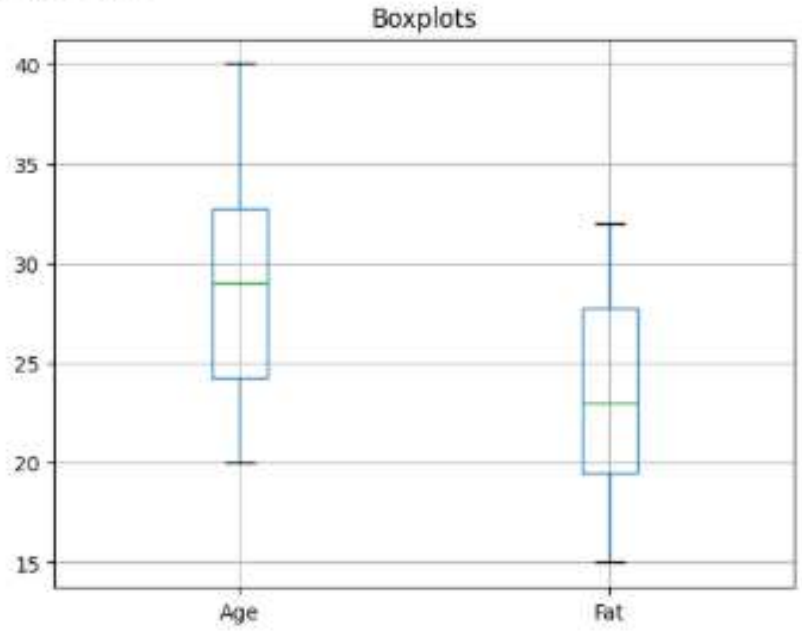
print("Standard deviation:")
print(data.std())

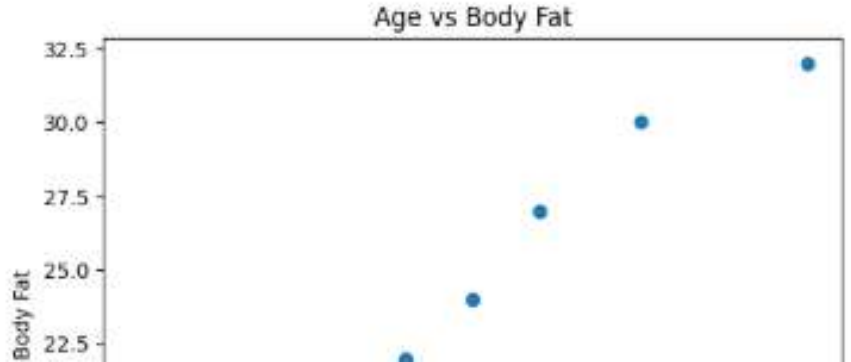
data[["Age", "Fat"]].boxplot()
plt.title("Boxplots")
plt.show()

plt.scatter(data["Age"], data["Fat"])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(data["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Age")
plt.show()
```

```
Mean:
Age    29.0
Fat    23.5
dtype: float64
Median:
Age    29.0
Fat    23.0
dtype: float64
Standard deviation:
Age    6.697548
Fat    5.903994
dtype: float64
```





```
import scipy.stats as stats

data = pd.DataFrame({
    "Age": [20, 22, 25, 28, 30, 32, 35, 40],
    "Fat": [15, 18, 20, 22, 24, 27, 30, 32]
})

print("Mean:")
print(data.mean())

print("Median:")
print(data.median())

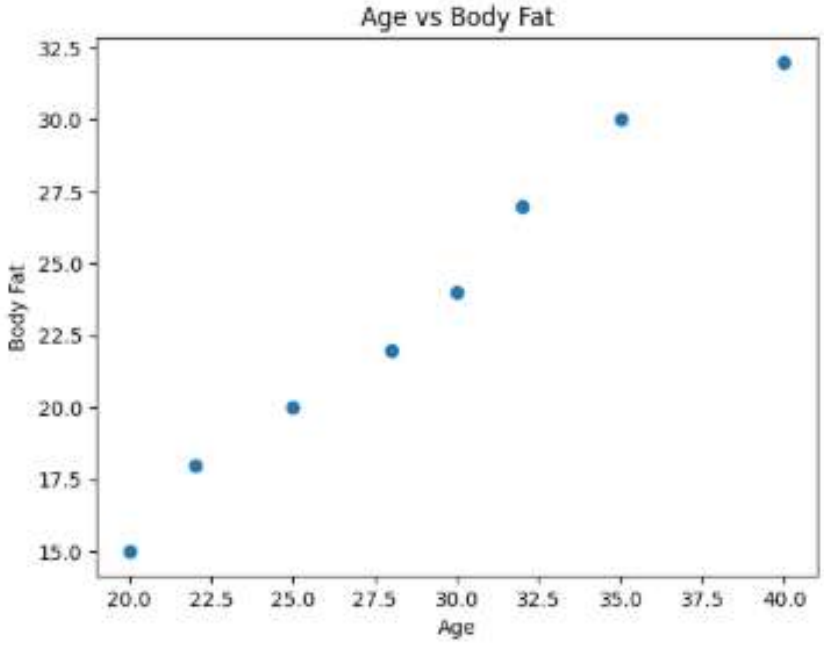
print("Standard deviation:")
print(data.std())

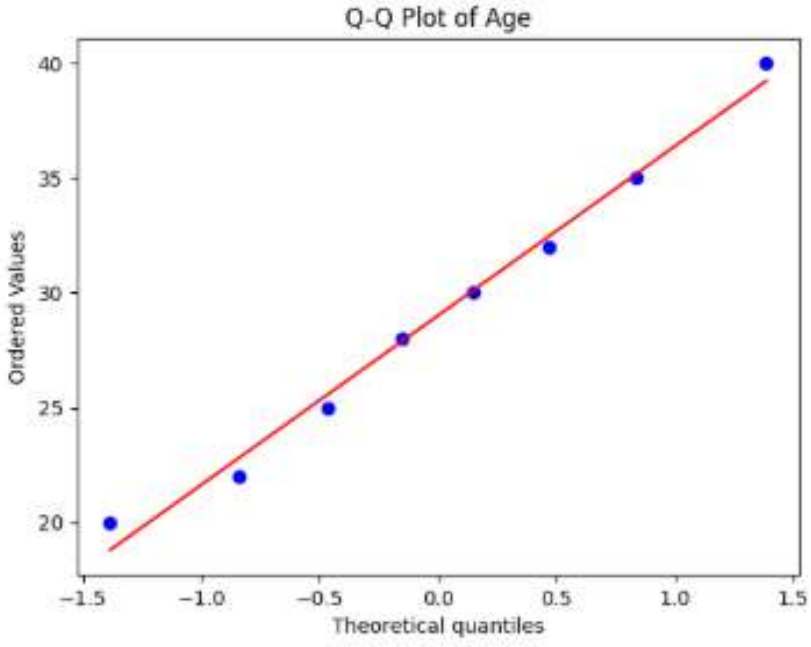
data[["Age", "Fat"]].boxplot()
plt.title("Boxplots")
plt.show()

plt.scatter(data["Age"], data["Fat"])
plt.xlabel("Age")
plt.ylabel("Body Fat")
plt.title("Age vs Body Fat")
plt.show()

stats.probplot(data["Age"], dist="norm", plot=plt)
plt.title("Q-Q Plot of Age")
plt.show()
```







```python
import numpy as np
from scipy import stats

drug = np.array([8, 10, 12, 9, 11, 13, 10, 12])
placebo = np.array([3, 5, 4, 6, 2, 5, 4, 3])

def confidence_interval(data):
    mean = np.mean(data)
    sem = stats.sem(data)
    interval = stats.t.interval(
        0.95, len(data) - 1, loc=mean, scale=sem
    )
    return interval

print("Drug 95% CI:", confidence_interval(drug))
print("Placebo 95% CI:", confidence_interval(placebo))
```

```
Drug 95% CI: (np.float64(9.216289685290807), np.float64(12.033710314709193))
Placebo 95% CI: (np.float64(2.9053916697328868), np.float64(5.094608330267113))
```

```python
import numpy as np
from scipy.stats import ttest_ind

design_A = np.array([0.10, 0.12, 0.11, 0.13, 0.09, 0.12])
design_B = np.array([0.14, 0.15, 0.13, 0.16, 0.12, 0.14])

t_stat, p_value = ttest_ind(design_A, design_B)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Statistically significant difference")
else:
    print("No statistically significant difference")
```

```
t-statistic = -3.3999999999999995
p-value = 0.006771072424337956
Statistically significant difference
```

```python
import pandas as pd
import numpy as np
from scipy import stats

# Programmatically create a mock rare_elements.csv to prevent FileNotFoundError
mock_elements = pd.DataFrame({
    "Concentration": [12.5, 14.2, 11.8, 15.1, 13.9, 12.0, 16.5, 14.8, 13.0, 15.5]
})
mock_elements.to_csv("rare_elements.csv", index=False)

data = pd.read_csv("rare_elements.csv")
values = data.iloc[:, 0].dropna()

n = int(input("Enter sample size: "))
confidence = float(input("Enter confidence level (for example 0.95): "))
precision = float(input("Enter desired precision: "))

# Adjust confidence level if entered as a percentage (e.g., 95 instead of 0.95)
if confidence > 1:
    confidence = confidence / 100.0

sample = values.sample(n=min(n, len(values)), random_state=1)

mean = sample.mean()
sem = stats.sem(sample)

ci = stats.t.interval(
    confidence, len(sample) - 1, loc=mean, scale=sem
)

print("Point estimate =", mean)
print("Confidence interval =", ci)
print("Desired precision =", precision)
```

```
Enter sample size: 5
Enter confidence level (for example 0.95): 11.8
Enter desired precision: 0.7
Point estimate = 14.039999999999997
Confidence interval = (np.float64(13.900359697055015), np.float64(14.17964030294498))
Desired precision = 0.7
```

```python
import pandas as pd
from scipy import stats

# Programmatically create a mock customer_reviews.csv file to resolve the FileNotFoundError
mock_reviews = pd.DataFrame({
    "rating": [4.5, 4.0, 5.0, 3.5, 4.2, 4.8, 3.9, 4.6, 4.1, 4.7]
})
mock_reviews.to_csv("customer_reviews.csv", index=False)

data = pd.read_csv("customer_reviews.csv")
ratings = data["rating"].dropna()

confidence = 0.95
mean = ratings.mean()
sem = stats.sem(ratings)

ci = stats.t.interval(
    confidence, len(ratings) - 1,
    loc=mean, scale=sem
)

print("Mean rating =", mean)
print("95% Confidence interval =", ci)
```

```
Mean rating = 4.33
95% Confidence interval = (np.float64(3.996081626528685), np.float64(4.663918373471315))
```

```python
import numpy as np
from scipy.stats import ttest_ind
import matplotlib.pyplot as plt

control = np.array([5, 6, 7, 5, 6, 4, 7])
treatment = np.array([8, 9, 10, 9, 11, 8, 10])

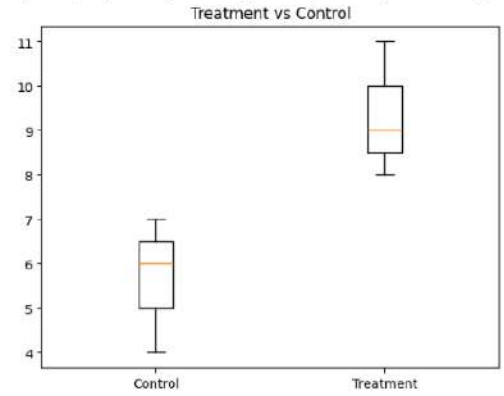
t_stat, p_value = ttest_ind(treatment, control)

print("t-statistic =", t_stat)
print("p-value =", p_value)

if p_value < 0.05:
    print("Significant difference")
else:
    print("No significant difference")

plt.boxplot([control, treatment], labels=["Control", "Treatment"])
plt.title("Treatment vs Control")
plt.show()
```

```
t-statistic = 6.004805767690768
p-value = 6.171074121437904e-05
Significant difference
/tmp/ipykernel_2296/4064839176.py:18: MatplotlibDeprecationWarning: The 'labels' parameter of boxplot() has been renamed 'tick_labels' since Matplotlib 3.9; support for the old name will be dropped in 3.11.
  plt.boxplot([control, treatment], labels=["Control", "Treatment"])
```



```python
from sklearn.neighbors import KNeighborsClassifier

# Features: symptom1, symptom2
X = [
    [1, 1], [1, 2], [2, 1],
    [8, 8], [9, 8], [8, 9]
]

# 0 = No condition, 1 = Condition
y = [0, 0, 0, 1, 1, 1]

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X, y)

patient = [[7, 8]]

prediction = model.predict(patient)

if prediction[0] == 1:
    print("Patient may have the condition")
else:
    print("Patient may not have the condition")
```

```
Patient may have the condition
```

```python
from sklearn.datasets import load_iris
from sklearn.tree import DecisionTreeClassifier

iris = load_iris()

X = iris.data
y = iris.target

model = DecisionTreeClassifier(random_state=1)
model.fit(X, y)

sepal_length = float(input("Sepal length: "))
sepal_width = float(input("Sepal width: "))
petal_length = float(input("Petal length: "))
petal_width = float(input("Petal width: "))

new_flower = [[
    sepal_length, sepal_width,
    petal_length, petal_width
]]

prediction = model.predict(new_flower)

print("Predicted species:", iris.target_names[prediction[0]])
```

```
Sepal length: 8
Sepal width: 5
Petal length: 7
Petal width: 6
Predicted species: virginica
```

```python
import numpy as np
from sklearn.linear_model import LinearRegression

# Area in square feet
X = np.array([[1000], [1200], [1500], [1800], [2000]])
# House prices
y = np.array([200000, 240000, 300000, 360000, 400000])

model = LinearRegression()
model.fit(X, y)

area = float(input("Enter house area: "))

prediction = model.predict([[area]])

print("Predicted house price =", prediction[0])
```

```
Enter house area: 25
Predicted house price = 4999.999999999944
```

```python
import numpy as np
from sklearn.linear_model import LogisticRegression

# Features: usage minutes, contract months
X = np.array([
    [100, 24], [120, 24], [80, 12],
    [300, 6], [280, 6], [350, 3]
])

# 0 = Not churned, 1 = Churned
y = np.array([0, 0, 0, 1, 1, 1])

model = LogisticRegression()
model.fit(X, y)

usage = float(input("Usage minutes: "))
contract = float(input("Contract duration: "))

prediction = model.predict([[usage, contract]])

if prediction[0] == 1:
    print("Customer may churn")
else:
    print("Customer may not churn")
```

```
Usage minutes: 7
Contract duration: 5
Customer may not churn
```

```python
import numpy as np
from sklearn.cluster import KMeans

X = np.array([
    [1000, 2],
    [1200, 3],
    [1100, 2],
    [5000, 10],
    [5200, 11],
    [4800, 9]
])

model = KMeans(n_clusters=2, random_state=1, n_init=10)
model.fit(X)

new_customer = [[1500, 3]]
cluster = model.predict(new_customer)

print("Customer belongs to cluster:", cluster[0])
```

```
Customer belongs to cluster: 0
```

```python
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

# Actual and predicted values
y_test = [0, 1, 1, 0, 1, 0, 1, 1]
y_pred = [0, 1, 0, 0, 1, 0, 1, 0]

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))
```

```
Accuracy = 0.75
Precision = 1.0
Recall = 0.6
F1-score = 0.75
```

```python
import numpy as np
from sklearn.tree import DecisionTreeRegressor

# Features: mileage, age
X = np.array([
    [20000, 2], [30000, 3], [40000, 4],
    [60000, 5], [80000, 7], [100000, 9]
])

price = np.array([18000, 17000, 15000, 13000, 10000, 7000])

model = DecisionTreeRegressor(max_depth=3, random_state=1)
model.fit(X, price)

mileage = float(input("Mileage: "))
age = float(input("Age: "))

prediction = model.predict([[mileage, age]])

print("Predicted price =", prediction[0])
```

```
Mileage: 20
Age: 18
Predicted price = 7000.0
```

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

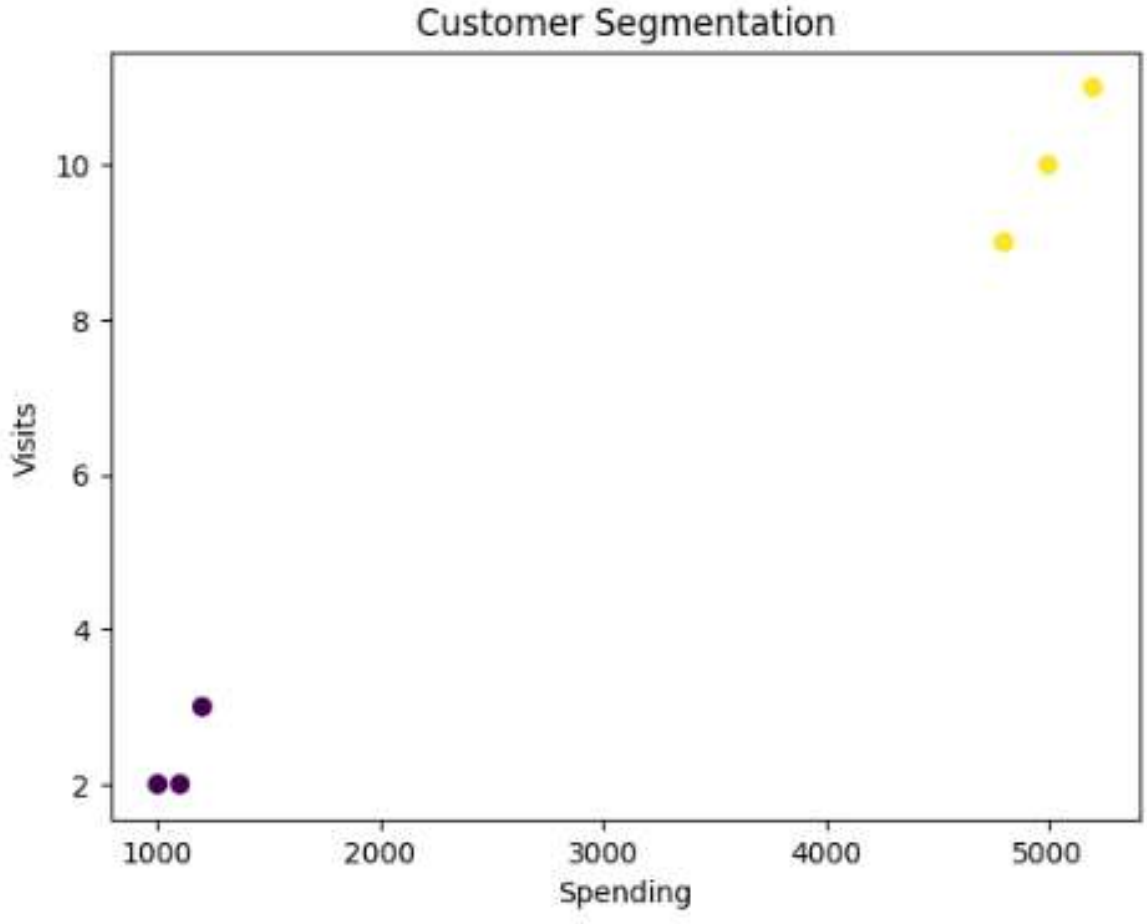
data = pd.DataFrame({
    "Spending": [1000, 1200, 1100, 5000, 5200, 4800],
    "Visits": [2, 3, 2, 10, 11, 9]
})

model = KMeans(n_clusters=2, random_state=1, n_init=10)
data["Cluster"] = model.fit_predict(
    data[["Spending", "Visits"]]
)

print(data)

plt.scatter(
    data["Spending"],
    data["Visits"],
    c=data["Cluster"]
)
plt.xlabel("Spending")
plt.ylabel("Visits")
plt.title("Customer Segmentation")
plt.show()
```

```
   Spending  Visits  Cluster
0      1000       2        0
1      1200       3        0
2      1100       2        0
3      5000      10        1
4      5200      11        1
5      4800       9        1
```



```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Size": [800, 1000, 1200, 1500, 1800, 2000],
    "Price": [160, 200, 240, 300, 360, 400]
})

X = data[["Size"]]
y = data["Price"]

model = LinearRegression()
model.fit(X, y)

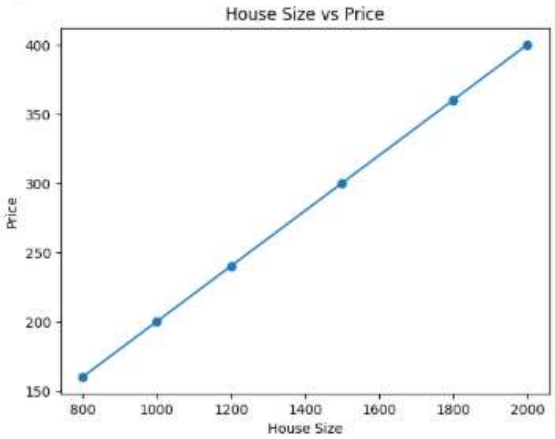
predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))

plt.scatter(data["Size"], data["Price"])
plt.plot(data["Size"], predicted)
plt.xlabel("House Size")
plt.ylabel("Price")
plt.title("House Size vs Price")
plt.show()

new_size = float(input("Enter house size: "))
print("Predicted price =", model.predict([[new_size]])[0])
```

```
R-squared = 1.0
```




```
Enter house size: 24
Predicted price = 4.799999999999945
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

```python
import pandas as pd
from sklearn.linear_model import LinearRegression
from sklearn.metrics import r2_score

data = pd.DataFrame({
    "Engine": [1.2, 1.5, 1.8, 2.0, 2.2, 2.5],
    "Horsepower": [80, 100, 120, 140, 160, 190],
    "Mileage": [20, 18, 16, 15, 13, 11],
    "Price": [8000, 10000, 13000, 16000, 19000, 23000]
})

X = data[["Engine", "Horsepower", "Mileage"]]
y = data["Price"]

model = LinearRegression()
model.fit(X, y)

predicted = model.predict(X)

print("R-squared =", r2_score(y, predicted))
print("Coefficients =", model.coef_)

new_car = [[1.8, 120, 16]]
print("Predicted price =", model.predict(new_car)[0])
```

```
R-squared = 0.9983049479376866
Coefficients = [-2692.30769231   180.76923077   115.38461538]
Predicted price = 12961.538461538466
/usr/local/lib/python3.13/dist-packages/sklearn/utils/validation.py:2739: UserWarning: X does not have valid feature names, but LinearRegression was fitted with feature names
  warnings.warn(
```

```python
import numpy as np
from sklearn.model_selection import train_test_split
from sklearn.neighbors import KNeighborsClassifier
from sklearn.metrics import (
    accuracy_score, precision_score,
    recall_score, f1_score
)

X = np.array([
    [20, 120], [22, 130], [25, 125], [27, 140],
    [40, 180], [42, 190], [45, 185], [48, 200]
])

y = np.array([0, 0, 0, 0, 1, 1, 1, 1])

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.25, random_state=1
)

model = KNeighborsClassifier(n_neighbors=3)
model.fit(X_train, y_train)

y_pred = model.predict(X_test)

print("Accuracy =", accuracy_score(y_test, y_pred))
print("Precision =", precision_score(y_test, y_pred))
print("Recall =", recall_score(y_test, y_pred))
print("F1-score =", f1_score(y_test, y_pred))

new_patient = [[30, 150]]
print("Prediction =", model.predict(new_patient))
```

```
Accuracy = 1.0
Precision = 1.0
Recall = 1.0
F1-score = 1.0
Prediction = [0]
```

```python
import pandas as pd
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [500, 700, 600, 4000, 4500, 4200, 1500, 1700],
    "Visits": [2, 3, 2, 10, 12, 11, 5, 6]
})

model = KMeans(n_clusters=3, random_state=1, n_init=10)

data["Cluster"] = model.fit_predict(
    data[["Spending", "Visits"]]
)

print(data)

print("Cluster centers:")
print(model.cluster_centers_)
```

```
   Spending  Visits  Cluster
0       500       2        1
1       700       3        1
2       600       2        1
3      4000      10        0
4      4500      12        0
5      4200      11        0
6      1500       5        2
7      1700       6        2
Cluster centers:
[[4.23333333e+03 1.10000000e+01]
 [6.00000000e+02 2.33333333e+00]
 [1.60000000e+03 5.50000000e+00]]
```

```python
import pandas as pd
import matplotlib.pyplot as plt
import numpy as np

# Programmatically create a mock stock_data.csv file to resolve the FileNotFoundError
np.random.seed(42)
mock_prices = 100 + np.cumsum(np.random.normal(0.1, 1.5, 100)) # Random walk
mock_data = pd.DataFrame({
    "Close": mock_prices
})
mock_data.to_csv("stock_data.csv", index=False)

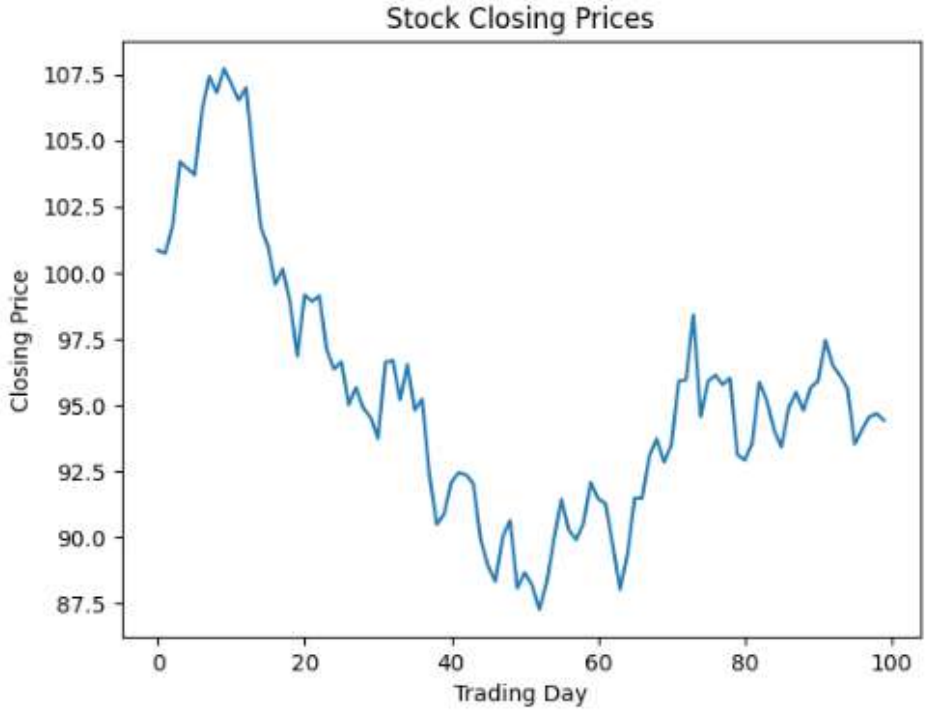
# Read the newly created dataset
data = pd.read_csv("stock_data.csv")

prices = data["Close"]

print("Mean =", prices.mean())
print("Standard deviation =", prices.std())
print("Minimum =", prices.min())
print("Maximum =", prices.max())
print("Range =", prices.max() - prices.min())

plt.plot(prices)
plt.title("Stock Closing Prices")
plt.xlabel("Trading Day")
plt.ylabel("Closing Price")
plt.show()
```

```
Mean = 95.44222630614448
Standard deviation = 4.96024941695371
Minimum = 87.28257663879828
Maximum = 107.72091667548136
Range = 20.438340036683087
```



```python
import pandas as pd
import matplotlib.pyplot as plt

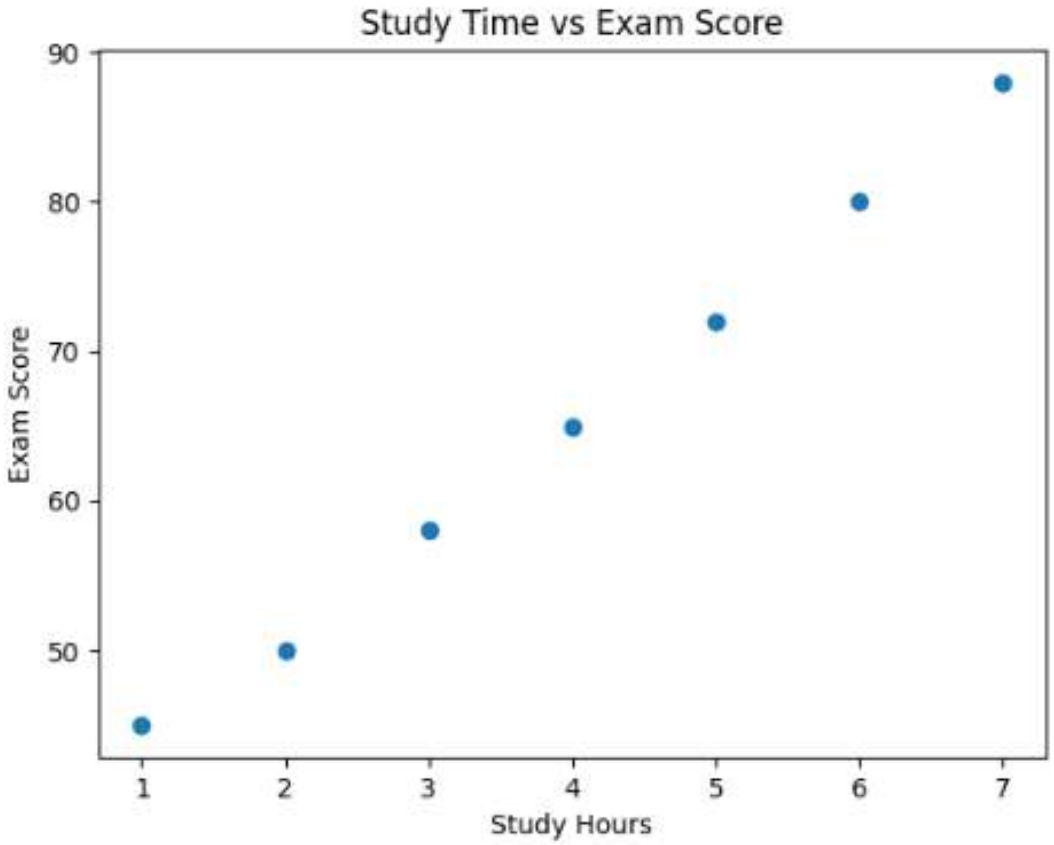
data = pd.DataFrame({
    "Study_Hours": [1, 2, 3, 4, 5, 6, 7],
    "Score": [45, 50, 58, 65, 72, 80, 88]
})

correlation = data["Study_Hours"].corr(data["Score"])

print("Correlation =", correlation)

plt.scatter(data["Study_Hours"], data["Score"])
plt.xlabel("Study Hours")
plt.ylabel("Exam Score")
plt.title("Study Time vs Exam Score")
plt.show()
```

```
Correlation = 0.9986559215086923
```



```python
import numpy as np

# Rows = cities, columns = daily readings
temperatures = np.array([
    [25, 27, 30, 28, 26],
    [20, 21, 22, 21, 20],
    [30, 32, 35, 33, 31]
])

cities = ["Chennai", "Bengaluru", "Delhi"]

mean = np.mean(temperatures, axis=1)
std = np.std(temperatures, axis=1)
temperature_range = np.ptp(temperatures, axis=1)

for i in range(3):
    print(cities[i])
    print("Mean =", mean[i])
    print("Standard deviation =", std[i])
    print("Range =", temperature_range[i])

highest_range = np.argmax(temperature_range)
most_consistent = np.argmin(std)

print("Highest temperature range:", cities[highest_range])
print("Most consistent city:", cities[most_consistent])
```

```
Chennai
Mean = 27.2
Standard deviation = 1.7204650534085253
Range = 5
Bengaluru
Mean = 20.8
Standard deviation = 0.7483314773547882
Range = 2
Delhi
Mean = 32.2
Standard deviation = 1.7204650534085253
Range = 5
Highest temperature range: Chennai
Most consistent city: Bengaluru
```

```python
import pandas as pd
import matplotlib.pyplot as plt
from sklearn.cluster import KMeans

data = pd.DataFrame({
    "Spending": [500, 600, 550, 3000, 3200, 3100, 1500, 1600],
    "Items": [2, 3, 2, 10, 11, 9, 5, 6]
})

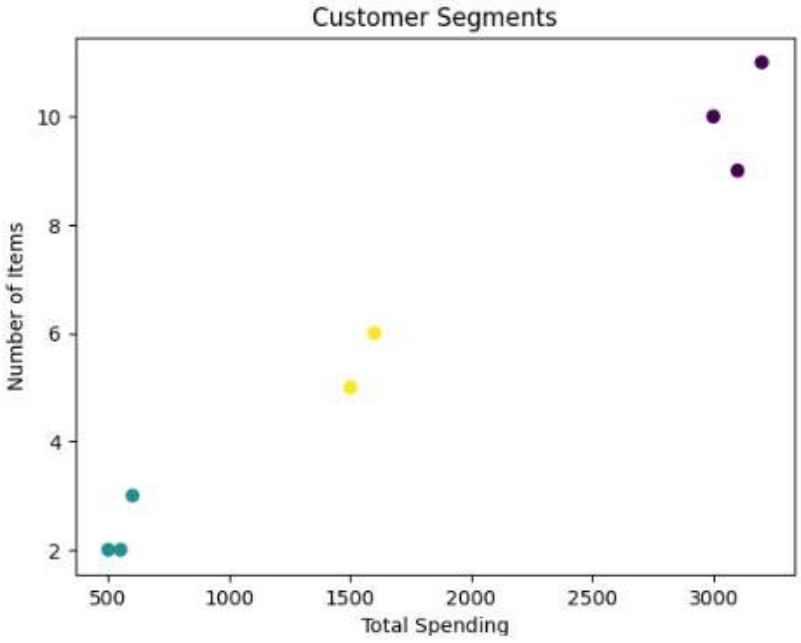
model = KMeans(n_clusters=3, random_state=1, n_init=10)

data["Cluster"] = model.fit_predict(
    data[["Spending", "Items"]]
)

print(data)

plt.scatter(
    data["Spending"],
    data["Items"],
    c=data["Cluster"]
)
plt.xlabel("Total Spending")
plt.ylabel("Number of Items")
plt.title("Customer Segments")
plt.show()
```

```
   Spending  Items  Cluster
0       500      2        1
1       600      3        1
2       550      2        1
3      3000     10        0
4      3200     11        0
5      3100      9        0
6      1500      5        2
7      1600      6        2
```



```python
import pandas as pd
import matplotlib.pyplot as plt

# Create and save the dataset
data = pd.DataFrame({
    "Name": ["A", "B", "C", "D", "E", "F", "G"],
    "Age": [22, 25, 28, 24, 30, 27, 23],
    "Position": ["Forward", "Midfielder", "Forward",
                 "Defender", "Goalkeeper", "Forward", "Defender"],
    "Goals": [15, 8, 20, 4, 1, 12, 5],
    "Salary": [5000, 4500, 7000, 3500, 4000, 6000, 3800]
})

data.to_csv("soccer_players.csv", index=False)

# Read CSV
players = pd.read_csv("soccer_players.csv")

print("Top 5 goal scorers:")
print(players.nlargest(5, "Goals")[["Name", "Goals"]])

print("\nTop 5 salaries:")
print(players.nlargest(5, "Salary")[["Name", "Salary"]])

average_age = players["Age"].mean()
print("\nAverage age =", average_age)

print("\nPlayers above average age:")
print(players[players["Age"] > average_age]["Name"])

players["Position"].value_counts().plot(kind="bar")
plt.title("Players by Position")
plt.xlabel("Position")
plt.ylabel("Number of Players")
plt.show()
```

```
Top 5 goal scorers:
  Name  Goals
2    C     20
0    A     15
5    F     12
1    B      8
6    G      5

Top 5 salaries:
  Name  Salary
2    C    7000
5    F    6000
0    A    5000
1    B    4500
4    E    4000

Average age = 25.571428571428573

Players above average age:
2    C
4    E
5    F
Name: Name, dtype: object
```

```
Top 5 goal scorers:
  Name  Goals
2    C     20
0    A     15
5    F     12
1    B      8
6    G      5

Top 5 salaries:
  Name  Salary
2    C    7000
5    F    6000
0    A    5000
1    B    4500
4    E    4000

Average age = 25.571428571428573

Players above average age:
2    C
4    E
5    F
Name: Name, dtype: object
```

